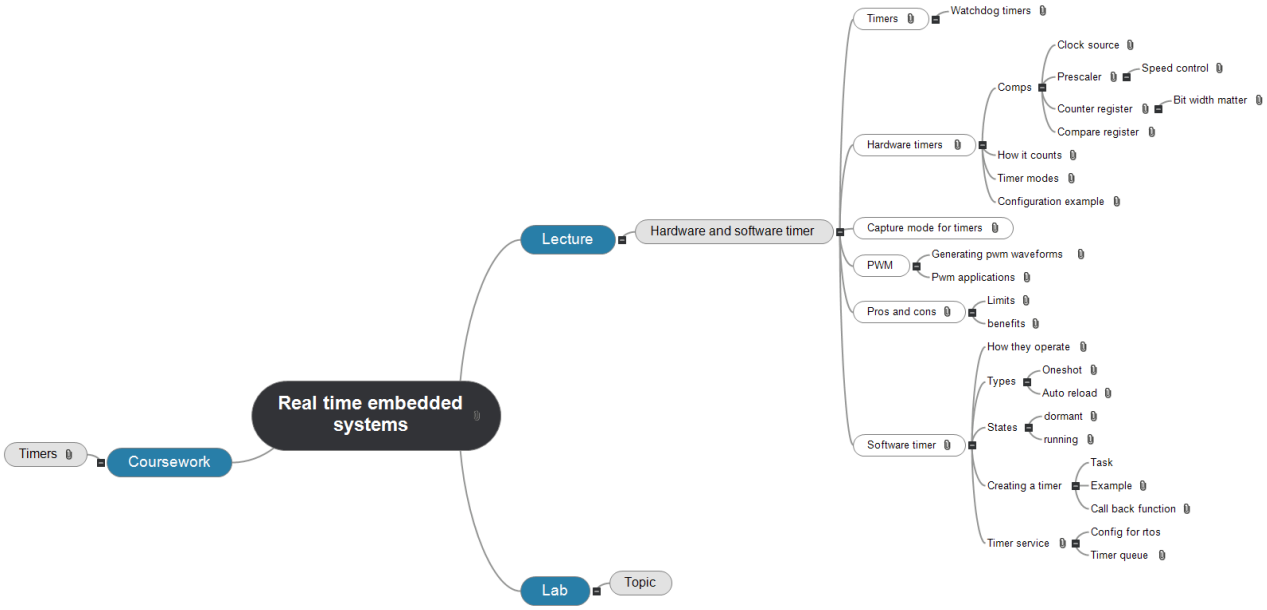
REAL TIME EMBEDDED SYSTEMS

REAL TIME EMBEDDED SYSTEMS



Lecture 3

Hardware and software timer 3

Timers 3

Hardware timers 4

Capture mode for timers 8

PWM 8

Pros and cons 8

Software timer 9

Lab 11

Topic 11

Coursework 11

Timers 11

By alex

Power point https://ports-main-mdl-euwest2.s3.eu-west-2.amazonaws.com/f6/d7/f6d7402145d6702630bf8eeb8d365c4df83cb823?response-content-disposition=inline%3B%20filename%3D%22Timer_verified.pdf%22&response-content-type=application%2Fpdf&X-Amz-Content-Sha256=UNSIGNED-PAYLOAD&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAWRN6GJFLZ774IX4M%2F20260319%2Feu-west-2%2Fs3%2Faws4_request&X-Amz-Date=20260319T103743Z&X-Amz-SignedHeaders=host&X-Amz-Expires=21557&X-Amz-Signature=6ba4724d8ebc6297202cc83d519281594772d96bd14b6ede6e50797704e787bb

Or see

My github here() i will check the power points and take bits to work as assistance

1 LECTURE

1.1 HARDWARE AND SOFTWARE TIMER

1.1.1 TIMERS

Why

Timers act as heart beats for the system enabling systems to accurately respond to time dependent systems

Embedded

Abs breaks wheel speed measurements have a precision of microseconds.

Smart thermo stats have an accuracy of seconds.

These are examples we should understand that not all timers need same precision.

Approach 1

While loops / for loops these waste time and cpus are locked in

Apporach 2

Hardware timer + interupt this leaves the CPU to do what it wants while the function is interrupted.

Most systems have a watchdog timer this timer resets the entier cpu/mcu if its ever locked or stuck this timer simply expects to be updated if its not then it will reset it does not care that you are doing something as such you add time to this timer regularly for operating systems this is less of an issue as it handles the watchdog timer.

1.1.1.1 WATCHDOG TIMERS

https://en.wikipedia.org/wiki/Mars_Pathfinder

https://en.wikipedia.org/wiki/Priority_inversion

this is a bug that was caused by a poorly designed watch dog timer

1.1.2 HARDWARE TIMERS

Their are multiple types of hardware timer depending on process and processor

For esp32 their are two types

Watchdog

General purpose

You can use the general purpose but not the watchdog for processing generally.

Hardware timers have 4 parts

Clock source

Prescaler

Counter register

Compare register

1.1.2.1 COMPS

1.1.2.1.1 CLOCK SOURCE

Internal oscilator with a defiend freq

normaly an external crystal with a clock pin

1.1.2.1.2 PRESCALER

Divides the clock to a different rate normaly timer freq = cloc_freq/ prescaler.

For example we want 10khz 72MHz/7200 = 10khz so 0.1ms

1.1.2.1.2.1 SPEED CONTROL

1 division = same speed as the clock so by pass

if 72mhz

72 = 1mhz micorsecond timing

7200 = 10 khz millisecond events

72000 1khz second range timing

Define the fastest the system will ever need to respond and then decide the prescaler you need if its too fast your wasitng cpu time if your too slow you miss the deadline

1.1.2.1.3 COUNTER REGISTER

Counter register counts up or down on each tick

This could be 8 16 or 32 bits

1.1.2.1.3.1 BIT WIDTH MATTER

8 bit 256 bit count at prescaler of 1 it lasts 3.6 us 233ms

Counter width	Max count	Prescaler =1	Prescaler at 65536
8	256	3.6us	233 ms
16	65,536	0.91ms	59.6 s
32	4,294,967,296	59.6 s	~3.9 million s

a large number of mcus use arm cortext m mcus have 3 - 12 timers depending on the cor you pick.

TIM = Timer (so tim1 is timer 1)

STM32F103

TIM1,TIM8: 16bit advanced timers

TIM2,TIM3,TIM4: 16bit general purpose

SysTick: 24bit system timer used by rtos

1.1.2.1.4 COMPARE REGISTER

When the counter == the compare register we reset the counter register and trigger the hardware timer

1.1.2.2 HOW IT COUNTS

T period = (prescaler * (autoReload +1))/fclock

When the auto reload triggers it causes a pulse(i think bit confused by his explination)

1.1.2.3 TIMER MODES

Need to figure out which a processor is other wise the system will not work properly

Count up mode (most common)

0 ->1 ->2 and so in this one is easiest to use and very common

Count down mode

Center alligned mode (used for advanced PWM)

Counts to max compares if it matches the max and then counts down and monitors the pulse (think)

This

1.1.2.4 CONFIGURATION EXAMPLE

// configure TIM2 for stm32

RCC->APV1ENR |= RCC_APB1ENR_TIM2EN

TIM

1.1.3 CAPTURE MODE FOR TIMERS

This is used for measuring things so

Capture the rpm of a system

At each rising edge you capture a bit of data and then you can do understand the speed of the system(maybe again he lost me)

1.1.4 PWM

1.1.4.1 GENERATING PWM WAVEFORMS

Const uint8_t sien_table[64] ={128,140,153};

Volatile uint8_t sample_index =0;

1.1.4.2 PWM APPLICATIONS

Fast motors

Typical freqs between 10khz and 25khz

1.1.5 PROS AND CONS

use hardware timers where you need precision use software where you don't

1.1.5.1 LIMITS

Their are a limtied number of timers on each mcu so use with acre

Their complex to configure

Can be hard to manage

Each timer is more power and more pcb cost

1.1.5.2 BENEFITS

Precice timig with high resolution

Operates independently of cpu exectuion

1.1.6 SOFTWARE TIMER

Onr hardware timer can drive all software timers

Each software timer is just a data structure in memory

The rtos kernel checks expiry on each tick

Virtually unlimited in number(memory limited)

1.1.6.1 HOW THEY OPERATE

Tick interrupt fires every 1ms

Timere services wakes up

Timer 1 target tick 1500 , counter 1500 pulse high

1.1.6.2 TYPES

1.1.6.2.1 ONESHOT

Used onces and then it is dormant

1.1.6.2.2 AUTO RELOAD

Cyclic does not go dormant it is like an alarm clock

1.1.6.3 STATES

1.1.6.3.1 DORMANT

Does not check the call back again it is done and will not fire

1.1.6.3.2 RUNNING

Timer is counting down and will send call backs when requested.

1.1.6.4 CREATING A TIMER

1.1.6.4.1 TASK

1.1.6.4.2 EXAMPLE

```
TimmerHandle_t xLedTimer;


XledTimer = xTimerCreate(
                    "LEDTIMER",
                    pdMS_TO_TICKS(500,
                    pdTrue,
                    NULL,
                    vLedtimercallback,
);
```

1.1.6.4.3 CALL BACK FUNCTION

Keep it short and fast don't call delays

They are non blocking fucntions if they block then the system will miss its spec

1.1.6.5 TIMER SERVICE

Also know as the daemon task or timer service daemon

1.1.6.5.1 CONFIG FOR RTOS

1.1.6.5.2 TIMER QUEUE

Timer service commands

start timer

Stop timer

Reset timer

Change timer

2 LAB

2.1 TOPIC

3 COURSEWORK

3.1 TIMERS

Expect these to appear in the course work its something critical to the system